



EFFICIENT ARCHITECTURE FOR SCALABLE DATA PROCESSING

Prepared by :

Sharie Rhea

August 2026

Prepared For :

Dr. Maureen Kinkela

SNHU CS680 Module 5 Project 1

Table of Contents

Executive Summary.....	2
Background.....	3
Problem.....	3
Outcomes and Constraints.....	4
Algorithms and Data Structures.....	7
Welford's Method.....	7
Hash Maps.....	8
Inverted Indexing.....	8
SHA-256.....	9
B Trees and B+ Trees.....	9
Solutions.....	11
Challenge 1: Standard Deviation.....	11
Challenge 2: Word Frequency.....	11
Challenge 3: Checksum Validation.....	12
Challenge 4: Efficient Querying.....	13
Relevant Data.....	16
Limitations and Biases.....	17
Limitations.....	17
Biases.....	17
References.....	19

Executive Summary

ACME Neural Intelligence (ANI) is performing cutting-edge research in the field of general intelligence, and needs an efficient and scalable architecture to manipulate datasets and query data. As ANI's lead data analyst, I have prepared a set of recommended algorithms and data structures ideal for solving ANI's four main challenges, accompanied by a rationale for why these solutions represent the optimal approach for each task.

To determine the standard deviation of value across datasets, Welford's Method is an online algorithm that can be used to iteratively compute variance in a single pass during data ingestion. To quickly determine word frequency, a custom inverted index implemented with hash maps built during initial processing allows for near constant time lookups. Standard SHA-256 hashing should be used to validate each record and short-circuit further processing if the data integrity of a record has been violated. Finally, specific words can be queried using the same inverted index utilized for word frequency, and a B+ Tree can be used to effectively partition records based on their value field and access records both randomly and sequentially.

Background

Problem

ACME Neural Intelligence (ANI) requires an efficient and scalable solution to manipulate datasets and generate actionable insights that hold the key to developing general intelligence.

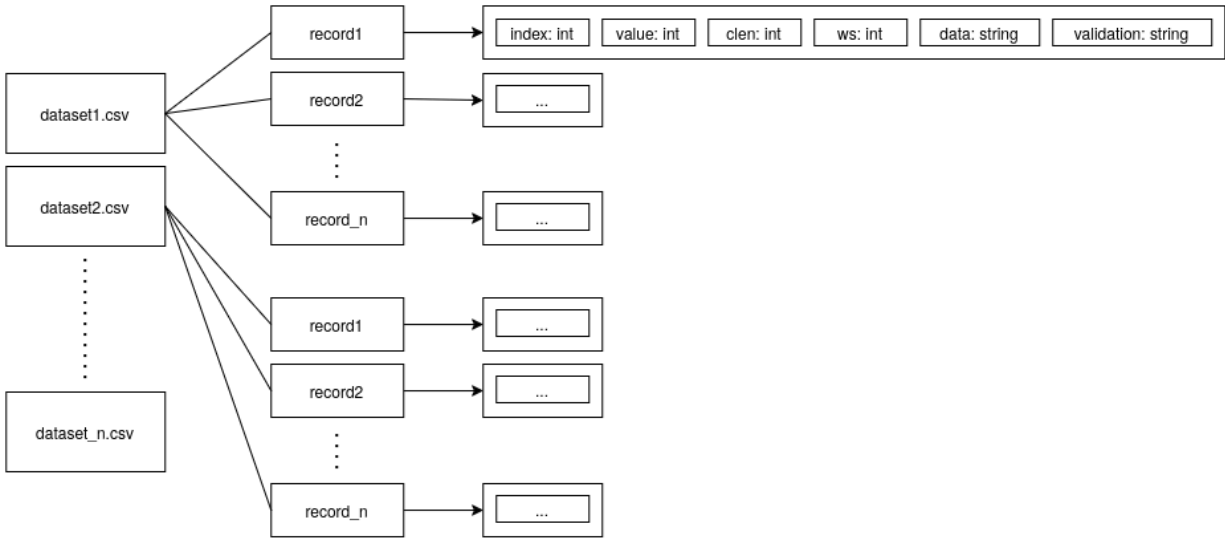
Each dataset contains one pipe-delimited record per line with the following 6 fields:

1. index: int - unique identifier for the specific record
2. value: int - based on ANI's proprietary algorithm
1. clen: int - length of the data field in characters (including white space)
2. ws: int - the number of spaces in the data field
3. data: string - list of words
4. validation: string - SHA-256 signature of all previous fields as a pipe-delimited string

Figure 1 illustrates the overall structure of the input data.

Figure 1

ANI's Dataset Format



ANI has identified 4 primary challenges that will aid in further research:

1. Determine the standard deviation for the value column across datasets.
2. Identify the frequency (number of occurrences) of a word in one or more datasets.
3. Validate records using the validation field, which is a checksum.
4. Efficiently query records by either value or data contents.

Outcomes and Constraints

ANI's main goals center around rapid solutions and low data access times. The largest constraint is that, due to the large scale of the data, it is assumed that not all information will fit into main memory at a single time. Under the assumption that ANI has access to plentiful secondary storage, their focus is centered around reducing redundant reads and heavily relies on processing data upfront. Beyond these general goals, each of the 4 challenges has unique success criteria and potential constraints.

The desired outcome for the first challenge—determining a single number for standard deviation of value across all datasets—is an accurate numerical figure that is computed efficiently. In this

context, efficiently means in $O(n)$ time or faster for n total records, as the naive approach requires two passes through the data. This ensures that the solution is viable at scale when hundreds of thousands or millions of records are present. Additionally, an algorithm with a high degree of parallelism is desired such that work may be distributed across multiple worker nodes, further enhancing efficiency.

The second challenge requires retrieving the frequency of a specific word for one or more datasets. The output of this function should return an integer or list of integers, depending on how many datasets were specified. For this task, retrieval with an asymptotic time complexity of $O(1)$ indicates a successful and scalable solution. A major constraint for this challenge is that it is not feasible to scan through the data in its entirety to calculate the frequency for each individual query. Instead, it must be computed during data ingestion and maintained as data is added.

Record validation via the record's existing checksum produces a binary outcome (true or false), which covers the third challenge. The algorithm used to verify the hash must be accurate and repeatable, ensuring that records that have lost their integrity are not used for training. This must be accomplished in constant time on any given record, for an overall time complexity of $O(n)$ for n total records.

The fourth and final challenge is to efficiently query for specific records based on either a value or word in the data field. For value-based queries, the output is a custom data structure such as `list[ pair( dataset, list[ record indices ] ) ]`. This structure ensures that each record retains its relationship to the dataset it is contained in. For word-based queries, the output is similar and has the structure `list[ pair( dataset, list[ pair( record index, position ) ] ) ]`. The difference is the addition of the position of the target word within the data field of the record. The naive approach of searching linearly is $O(n)$ for n records. ANI's threshold is a solution with a time complexity of less than $O(n)$, but the objective for this operation is $O(\log n)$ or faster.

Achieving these outcomes in an efficient and scalable manner is vital for ANI's groundbreaking research and progress in the field of general intelligence.

Algorithms and Data Structures

To solve ANI's data manipulation challenges, the intelligent use of various algorithms and data structures are required. This section highlights a selected group that will be utilized in the suggested solutions.

Welford's Method

Welford's Method is an online, iteration-based algorithm for calculating variance, which is the average of the squared differences from the mean of a set of numbers (Welford, 1962). The traditional method for computing variance is done in two passes—sum all the numbers to determine the mean, then calculate the difference of each element from the mean and square it. This approach is numerically stable but inefficient and must be completely redone each time a new item is added. Welford's Method instead “gets the corrected sum of squares for n values from the corrected sum of squares for the first $n-1$ ” (Welford, 1962). In other words, the corrected sum of squares is built one element at a time, based on the result of all the previous elements. Significantly, this means that only a single pass through the data is required. Welford's iterative method has a time complexity of $O(1)$ for each step, yielding a complexity of $O(n)$ for n total numbers (Chen, 2019).

Besides the advantage of a shorter time complexity, this algorithm also mitigates the catastrophic cancellation caused by finite computer precision in alternative online algorithms (Chen, 2019). The structure of Welford's Method also allows for a high degree of parallelization. Worker nodes can each be assigned to a subsection of the data and compute the variance of that subsection. Then, the results of each worker node are recombined to achieve the final output.

Hash Maps

Hash maps are a data structure that stores key-value pairs and provide fast lookups by hashing the keys (GeeksForGeeks, 2023). To locate a value, its key is hashed and used as the address of a bucket which holds any values whose keys hash to the same value (GeeksForGeeks, 2023). There are multiple approaches for storing records within buckets such as chaining using arrays or linked lists. Another method is to probe for the next open slot to avoid overly full buckets (GeeksForGeeks, 2023). Ultimately, the efficiency of a hash map depends on how well-distributed the data is over the hash buckets. A hash function that always hashes to the same few values will not significantly improve lookup time because a standard linear or binary search will need to be done on the bucket itself.

Inverted Indexing

An inverted index is a data structure typically used to efficiently retrieve documents containing a specific word or value (GeeksForGeeks, 2018). Instead of storing each document and the words it contains, the organization method is “inverted” such that the key is the word and its value is the list of documents containing that value (GeeksForGeeks, 2018). The value can also be customized or enhanced to include metadata such as the total term frequency or position within the document (GeeksForGeeks, 2018). An inverted index is a powerful tool often used in databases and search engines because it can be implemented using a hash map to provide much more efficient lookups than searching through each document individually (Zobel et al., 1998). An additional benefit is that an inverted index is inexpensive to update and maintain as new information is added (GeeksForGeeks, 2018). The drawback is that indexing in this manner incurs a lot of storage overhead for storing many keys and layers of indices (GeeksForGeeks, 2018).

SHA-256

SHA-256 is a cryptographic hashing algorithm that “is used to map a message of arbitrary length to a fixed-length message digest” (NIST, 2024). Secure hashing algorithms are one-way, meaning that the original value cannot be determined by reverse-engineering the hash value. For ANI’s use case, the SHA-2 version of SHA-256 is used to generate the validation field of each record, provided a checksum or digital signature. The importance of this is that the checksum can be used to verify that none of the record’s fields have been altered, as the hash value would no longer match the validation field. It should be noted that storing the checksum alongside the data values is not recommended, as it would be possible to modify record data, compute the checksum, and alter the checksum to match. This violates data integrity in a way that bypasses any possible checks. As of 2015 NIST, has introduced SHA-3 into its family of secure hashing algorithms, which is based on a different generation method than SHA-2 (Dworkin, 2025). SHA-2 remains secure against current known attacks (NIST, 2024).

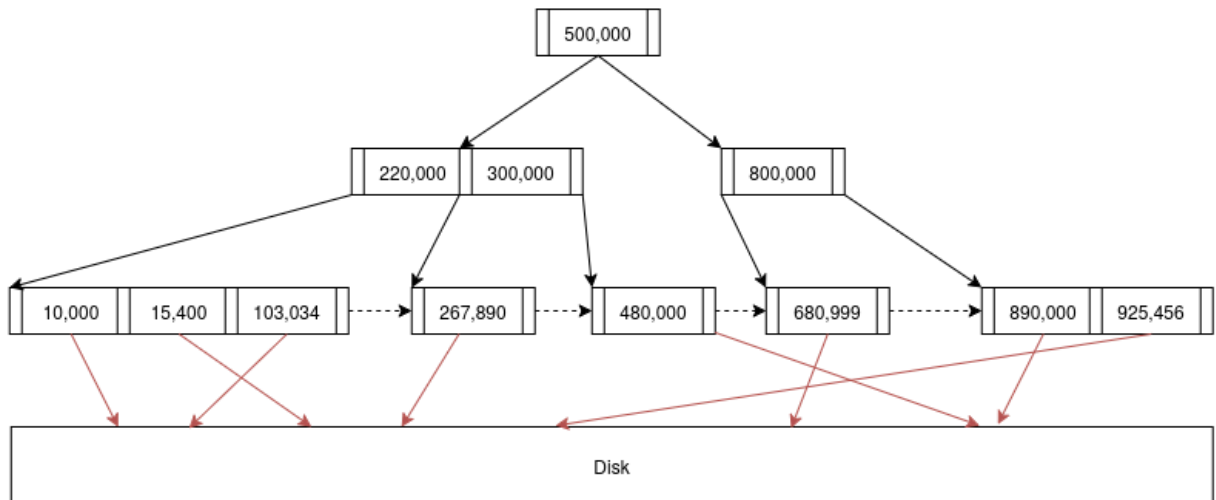
B Trees and B+ Trees

B Trees are a class of self-balancing search trees introduced by Bayer and McCreight (1970) in which each node can contain multiple keys and children, all leaf nodes reside at the same lowest level, and the tree height is minimized (Cormen et al., 2022; GeeksForGeeks, 2020). Due to their self-balancing nature, B Trees allow for standard retrieval, insertion, and deletion operations to be accomplished in $O(h)$ time, where h is the height of the tree (Cormen et al., 2022). While binary search trees already greatly improve the time to search for a specific value, B trees take this a step further by introducing multiple key comparisons at each node, where greater partitioning allows for a faster search (Cormen et al., 2022). This works well for random access, where any record may be queried at any time (Comer, 1979). However, B Trees do not allow for efficient sequential traversal (Comer, 1979). A variant of B Trees called B+ Trees

resolves this issue by only storing data in leaf nodes and adding pointers from each leaf node to the next, forming a linked list out of the lowest level of the tree (GeeksForGeeks, 2020). This linked list can be used to efficiently access data in order without tree traversal. Figure 2 shows an example of a B+ Tree for integers with each leaf node containing pointers to the data's location on disk. When large volumes of data need to be accessed, B+ Trees reduce the overall number of disk accesses by storing the search tree in memory and only requesting data from disk once its location is known (Comer, 1979).

Figure 2

B+ Tree for Integers with Pointers to Data Location on Disk



Solutions

Now that the problem and appropriate data structures and algorithms have been introduced, the recommended approach for each challenge will be outlined.

Challenge 1: Standard Deviation

To efficiently compute the standard deviation of the value field across datasets, Welford's Method can be used to track the mean and variance during data ingestion. Once data ingestion has been completed, standard deviation can be derived from these two values. Using this algorithm has the added benefit of numerical stability, parallelization, and extensibility. Due to ANI's large volume of data, multiple worker nodes, perhaps in a cloud or distributed architecture, can divide and conquer the task by computing the mean and variance of different portions of the total input, perhaps by dataset. Then, each intermediate value can be recombined in a main thread. As new data is collected and ingested, the existing mean and variance can be updated without having to recalculate or pass through data that has already been considered. Using this method with a single thread, the standard deviation can be calculated in $O(n)$ time for n records.

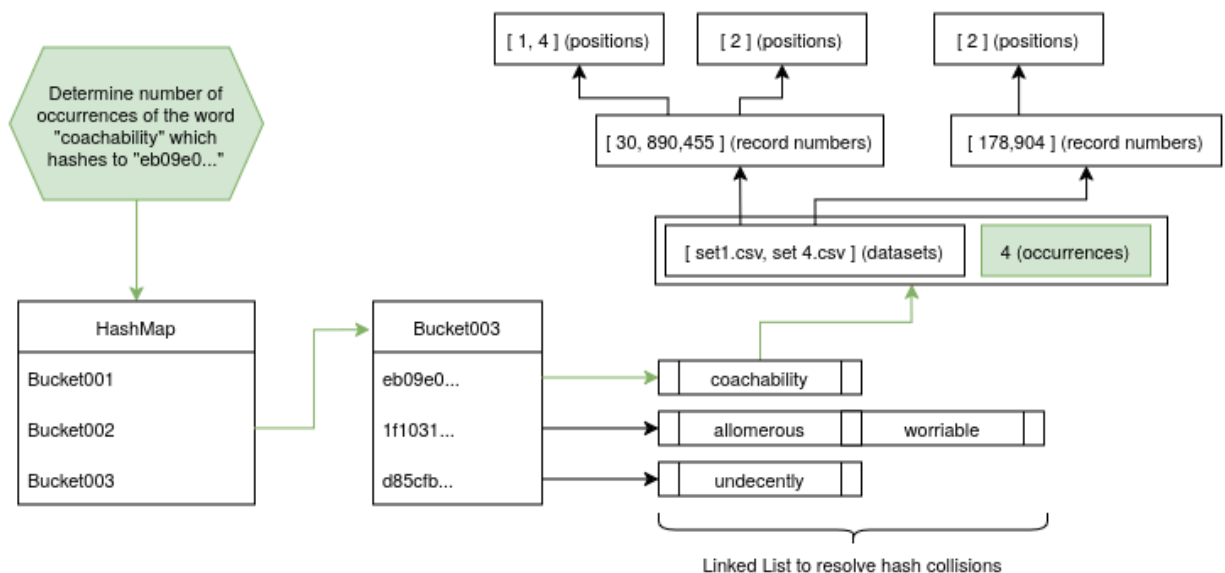
Challenge 2: Word Frequency

To rapidly determine the number of occurrences of a given word in one or more datasets, an inverted index implemented with a hash map provides near $O(1)$ lookups after the initial task of data structure creation (Zobel et al., 1998). Figure 3 shows the path for determining the overall frequency of the word "coachability" using one possible structure—note that values are for illustration purposes only. The recommended structure is to have a high-level hash map with buckets determined by the word's SHA-256 hash value and a further hash with a modulo operation to group into buckets. Within this bucket, an inverted index uses the word's SHA-256 hash to point towards a linked list to handle potential hash collisions. If necessary, the linked list

is searched sequentially, as hash collisions should be exceedingly rare. Once the word has been located, follow its pointer to a custom data structure storing a list of datasets this word appears in alongside its total number of occurrences. If the frequency in a particular dataset is desired, follow the dataset's pointer and take the length of the records list. This structure should be built during data ingestion and updated as each record is processed. If additional data is collected in the future, this structure is easy to maintain by following the same path as searching for each word in the record and updating values as needed (Zobel et al., 1998).

Figure 3

Determining the Word Frequency of "coachability"



Challenge 3: Checksum Validation

As with challenges one and two, the work for challenge three is performed during data ingestion. As each record is processed, the SHA-256 validation field is used to compare and ensure the data integrity of the record. This should be the first step, as there is no point in further processing the data if it is not valid. In the event that the checksum does not match, immediately continue to the next record. To compute the checksum, all fields except the

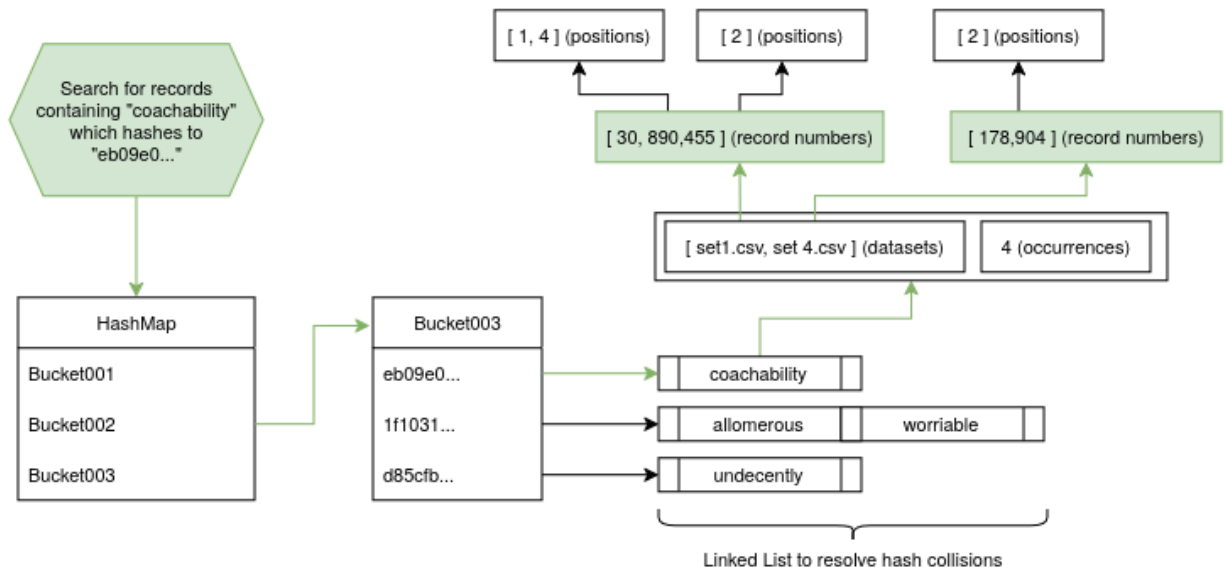
validation field are concatenated, separated by pipe characters. This string is then hashed using SHA-256 and compared with the validation field. The time complexity for hashing is typically constant, and therefore takes $O(n)$ time to process n records. As with Welford's Method, this work can easily be parallelized with multiple threads or workers since each record's validation check is independent from every other record.

Challenge 4: Efficient Querying

The final challenge is the most critical for ANI's general intelligence research, and has two subcomponents. This section first addresses word-based querying before detailing the solution for value fields. The data structure described in the solution for challenge two can be reutilized for this task by extending a few more layers of pointers. Using the same inverted index approach, Figure 4 illustrates the path to determine which datasets and records contain the word "coachability." As before, follow the hash into the appropriate bucket, determine the index within the bucket, and locate the word in the linked list. Following this pointer to the custom data structure, a list of each dataset this word appears in is stored, along with pointers to a list of record numbers. Each record number also stores a list of the word's position or positions in the record, in case that level of granularity is needed. By using the same data structure as challenge two, the initial processing workload during data ingestion is shared between the two tasks and is therefore more efficient.

Figure 4

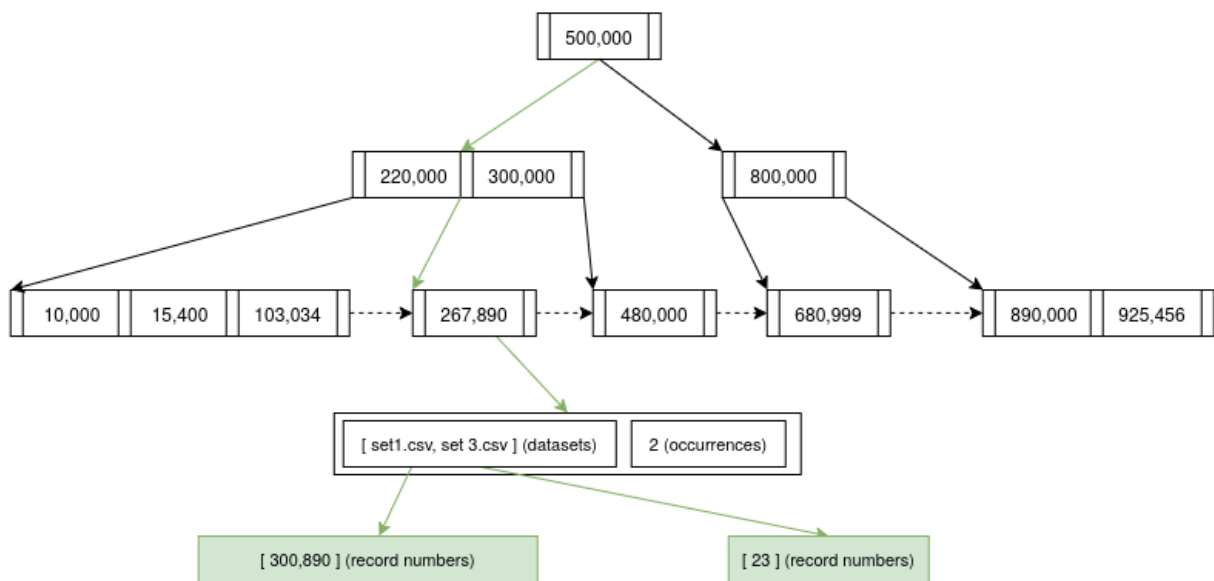
Determining the Records Containing "coachability"



Now the task of locating records with a particular value will be considered. Unfortunately, the inverted index cannot be reused for this task, as it is purely constructed for searching by word. For value, a different approach must be used. Analysis of the sample data provided by ANI shows that the range of the value field is 100,000 to 1,000,000, and is very evenly distributed. This even distribution is ideal for forming a B or B+ Tree, partitioning the data based on the value field itself, and may even be predetermined and structured before data ingestion to ensure a well-partitioned tree. For ANI's application, it may be desirable to pull all records with similar values—sequential access—or to access individual records—random access. Therefore, a B+ Tree is the recommended approach. Figure 5 showcases the path that would be followed to locate records with the value 267,890. As before with the inverted index, the final leaf nodes in the B+ Tree point to a custom data structure storing a list of datasets with pointers to a list of records numbers. Using this approach, the records for any given value can be accessed in $O(h)$ time, where h is the height of the B+ Tree. Additionally, using a B+ Tree means that fewer secondary storage or disk accesses are needed, since it is likely that the relatively small tree nodes and pointers may be stored in main memory.

Figure 5

Determining Records with the Value 267,890



Relevant Data

Some fields in ANI's record structure are relevant to the assigned tasks and some are not. For challenge one, only the value field is used to calculate the standard deviation. Further, since ANI is concerned with the standard deviation across datasets, the mean and variance for each individual dataset need not be stored. For the second task, the originating dataset, record index, data fields are the most relevant. An additional piece of metadata—the word's position—is also determined. The checksum and validation portion of data ingestion uses every field in the record, as the checksum must be computed from the concatenated string of the index, value, clen, ws, and data fields. Finally, the index, value, and data fields are most relevant for the task of querying by word or value. The clen (character length) and ws (white space) fields are auxiliary, as they are only used for the validation step. This is another reason for computing the checksum during validation, as these two fields do not need to be stored afterwards. If necessary, they can always be recomputed from the data field itself.

Limitations and Biases

Although the solutions recommended have been chosen with careful consideration, there are still limitations, implications, and biases that cannot be ignored.

Limitations

The main implication for each of the suggested solutions is that ANI is able to accept the significant upfront computational cost of building complex data structures during initial ingestion. Additionally, these structures incur overhead for storing multilevel indices and many pointers (GeeksForGeeks, 2018). The benefit is that future lookups are much more efficient than naive sequential searches, but the cost of secondary storage is still a consideration (Zobel et al., 1998). The approach recommended in this white paper follows the common assumption that in comparison to main memory, local or cloud disk storage is relatively plentiful, and the tradeoff between memory and time complexity is worthwhile (Cormen et al., 2022; GeeksForGeeks, 2018; GeeksForGeeks, 2023). However, this is still a major system limitation, as insufficient storage will cripple ANI's structure, and lack of compute power will slow data ingestion to a crawl.

Biases

The use of inverted indices to solve challenges two and four could potentially introduce bias by ignoring the potential benefits of tokenization or stemming to group similar words together (GeeksforGeeks, 2018). For general intelligence and machine learning applications—which is ANI's primary focus—it is often desirable to perform more general searches and clustering based on word roots, prefixes, or suffixes. The data structure recommended here does not take this into account. Additionally, the B+ Tree structure for task four assumes that the value field will always be well distributed. While B+ Trees do have a self-balancing nature, they will quickly

deteriorate if many duplicate values are present, creating long lists of datasets and record indices within a single leaf node (Bayer & McCreight, 1970; Comer, 1979).

Finally, the data structures and algorithms mentioned in this white paper have been chosen under the assumption that ANI prefers to construct its own solution to the common problem of searching vast volumes of data. Many commercial solutions, namely database management systems, exist to provide efficient queries as a service. Apache Spark, MySQL, MongoDB, Snowflake, Neo4j, and Amazon Web Services are all potential options for a commercial rather than in-house approach.

References

- Bayer, R., & McCreight, E. M. (1970). *Organization and maintenance of large ordered indices*.
<https://doi.org/10.1145/1734663.1734671>
- Chen, C. (2019, February 7). ChangyaoChen. *Welford algorithm for updating variance*.
<https://changyaochen.github.io/welford/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. The MIT Press.
- Comer, D. (1979). Ubiquitous B-Tree. *ACM Computing Surveys*, 11(2), 121–137.
<https://doi.org/10.1145/356770.356776>
- Dworkin, M. J. (2015). *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. <https://doi.org/10.6028/nist.fips.202>
- GeeksforGeeks. (2018). *Inverted Index*. In GeeksforGeeks.
<https://www.geeksforgeeks.org/dbms/inverted-index/>
- GeeksforGeeks. (2020). *Difference between B tree and B+ tree*. In GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/difference-between-b-tree-and-b-tree/>
- GeeksforGeeks. (2023). *Hashing in DBMS*. In GeeksforGeeks.
<https://www.geeksforgeeks.org/dbms/hashing-in-dbms/>
- NIST. (2024). *Hash Functions*. In Nist.gov. <https://csrc.nist.gov/projects/hash-functions>
- Welford, B. P. (1962). Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics*, 4(3), 419–420.
<https://doi.org/10.1080/00401706.1962.10490022>
- Zobel, J., Moffat, A., & Ramamohanarao, K. (1998). Inverted files versus signature files for text indexing. *ACM Transactions on Database Systems (TODS)*, 23(4), 453–490.
<https://doi.org/10.1145/296854.277632>